NPA – Exercise 2

Emily Seebeck	411364
Michel Schnitker	410230
Jarne Bärmann	409434
Raphael Krafczyk	393325

2025 – 11 – 02

Exercise 1

(a)

The NVIDIA Collective Communication Library (NCCL), often pronounced „Nickel“, defines five `Collective Communication Primitives` in its current documentation [1]. These primitives are listed below and are familiar from the lecture:

- AllReduce
- Broadcast
- Reduce
- AllGather
- ReduceScatter

The `Broadcast` command replaces the outdated `Bcast` command, which was previously implemented, thus creating a unified API [2].

(b)

1. `oneCCL`

The oneCCL is a Collective Communication Library provided by Intel, that focuses on deep learning applications, especially neural network training and the possibility to integrate oneCCL into deep learning frameworks, such as PyTorch [3]. oneCCL supports CPUs and GPUs as devices, but is specially optimized for Intel CPUs [4]. It is applicable for optimizing a single node with e.g. multiple CPUs and/or GPUs, but also for clusters of multiple nodes. The supported operations are the same as seen for NCCL in (a) [5].

2. `Gloo`

GLOO is an open source Collective Communication Library developed by Facebook/Meta. It is specialized for distributed machine learning and often used in e.g. PyTorch frameworks. It is especially useful, when the training is not only done on GPUs, as it supports CPU-only and mixed CPU-GPU tasks, while GPU-only tasks are also possible, while being not as efficient as NCCL [6].

In [7], Lee et al. state, that GLOO is most efficient, when small data exchanges between nodes are necessary.

3. `OCCL`

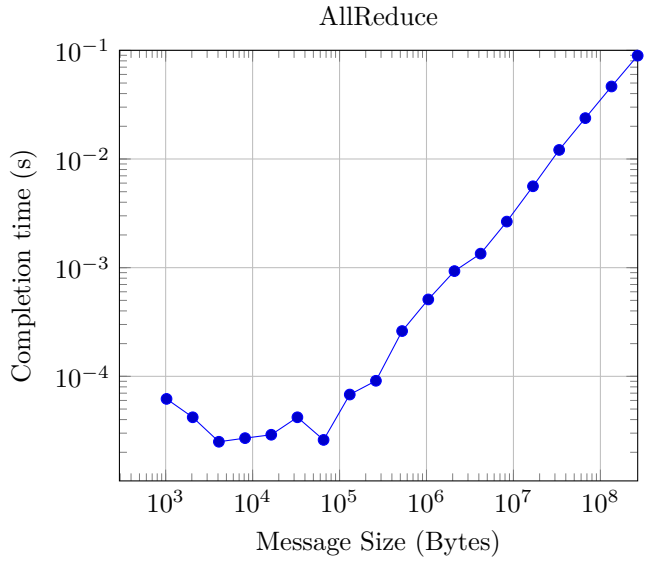
OCCL, whose name is not explicitly resolved, is a deadlock-free library for collective communication between GPUs. It is integrated into a distributed deep learning framework and demonstrated acceptable overheads compared to NCCL [8]. Due to the inherent asynchronicity of GPU cores and streams, collective operations on GPUs are prone to deadlocks. OCCL addresses this Problem by employing a global orchestration of collective calls that dynamically schedules operations across all participating devices [8]. The correctness and performance referred have not been verified by this Homework-Group.

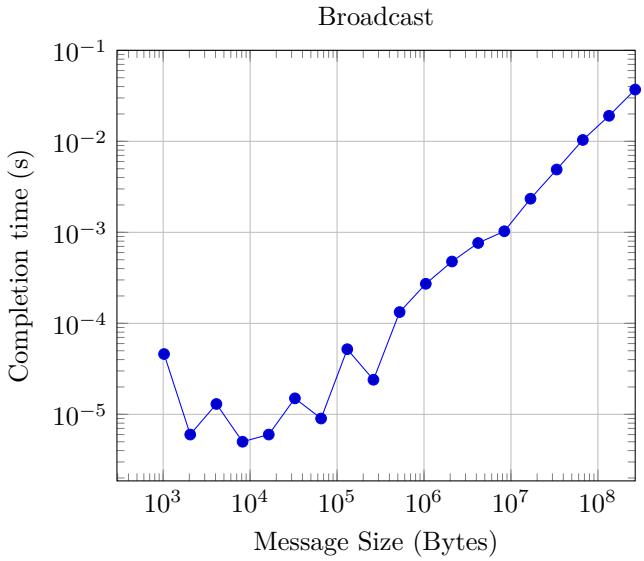
Exercise 2

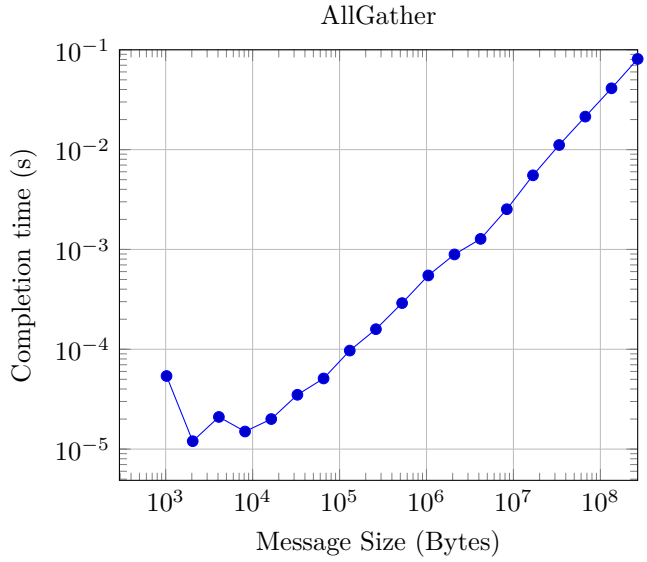
(a)

The data for the following plots has been generated with

```
mpiexec -n 2 python3 mpi-benchmark.py
```







In all plots, we can observe a linear trend (both axis scale logarithmically) of message size vs. completion time. This is consistent with the lecuture:

$$
\begin{aligned}
T_{\text{Reduce-Scatter}} &= T_{\text{AllGather}} \\
T_{\text{AllGather}} &= \log_2(n) \cdot \alpha + m \cdot \beta \\
T_{\text{AllReduce}} &= T_{\text{Reduce-Scatter}} + T_{\text{AllGather}} \\
T_{\text{Broadcast}} &= 2 \cdot \log_2(n) + 2 \cdot m \cdot \beta
\end{aligned}
$$

In all these formulas the message size m has a linear bandwidth factor, so we would expect the completion time to scale linearly with message size. As we can observe, it does.

Another interesting trend we observe is, that for all Graphs the message size of 1024 takes longer to complete

than 2048. This is also consistent across multiple runs, so this isn't a random fluctuation. In fact, this has to do with the algorithm that is selected by OpenMPI to facilitate the operation: Source[1].

Depending on the message size and number of processes, different algorithms get selected. Thus, the constant latency is higher or lower, depending on the algorithm used. This explains the decrease in completion time for higher message size.

(b)

1. `MPI.COMM_WORLD`

This is a static reference to a `Intracomm` object.

The purpose of this object is to facilitate communication between the spawned python processes.

2. `Get_rank()`

This method returns the rank of this process in a communicator. The rank is a "worker number", indicating which ID the current process has.

The purpose of this method is to determine the ID of the spawned thread in the group.

3. `Get_size()`

This method returns the number of processes in a communicator. If we execute
`mpiexec -n X python3 mpi-benchmark.py`, the output will be `X`.

The purpose of this method is to determine the number processes spawned.

4. `Barrier()`

The `Barrier()` function is a synchronization primitive. Hitting a `Barrier()` will block the caller until all processes have gotten to this point and called the `Barrier()`

The purpose of this function is to wait until all threads are ready.

5. `Allreduce()`

This is the AllReduce function we encountered during the Lecture.

This function takes three arguments:

```
sendbuf: BufSpec
recvbuf: BufSpec
op: Op
```

The `sendbuf` is the data from each process. It is combined with all others to form the result.

In the `recvbuf`, the result will be stored.

With `op` one can change the operation performed on the data.

References

[1] NVIDIA Corporation, *NVIDIA NCCL Documentation*, 2025. Accessed: 2025-11-01.

[2] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefler, "Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms," 2025.

[1] https://github.com/open-mpi/ompi/blob/3a2e90895e15822912002be1e9aea8032c4c0bae/ompi/mca/coll/tuned/coll_tuned_decision_fixed.c#L81-L215

[3] Intel Corporation, "Intel® oneapi collective communications library (oneccl)." https://www.intel.com/content/www/us/en/developer/tools/oneapi/oneccl.html, 2025. Accessed: 2025-11-02.

[4] oneAPI Specification (UXL Foundation), "oneccl concepts — main objects (oneapi specification)." https://oneapi-spec.uxlfoundation.org/specifications/oneapi/latest/elements/oneccl/source/spec/main_objects, 2025. Accessed: 2025-11-02.

[5] oneAPI Specification (UXL Foundation), "oneccl — introduction (oneapi specification)." https://oneapi-spec.uxlfoundation.org/specifications/oneapi/latest/elements/oneccl/source/introduction, 2025. Accessed: 2025-11-02.

[6] The ML Architect, "Communication protocols for distributed ml: Nccl, mpi and key patterns." https://themlarchitect.com/blog/communication-protocols-for-distributed-ml-nccl-mpi-and-key-patterns/, 2025. Accessed: 2025-11-02.

[7] S. Lee and J. Lee, "Collective communication performance evaluation for distributed deep learning training," *Applied Sciences*, vol. 14, no. 12, 2024.

[8] L. Pan, J. Liu, J. Yuan, R. Zhang, P. Li, and Z. Xiao, "Occl: a deadlock-free library for gpu collective communication," 2023.